Runtime Grounding:
ECMAScript Semantics for the SKJS Value Domain

Deep Dive A for "Refinement and Inference Growth in SKJS"

Models revision `bafdcf7c` of `main` (2026-08-05)

Abstract

The main soundness paper models the runtime as an abstract value domain Val equipped with predicates ($\approx$, truthy, callable, constructible, isArrayV, P_{inst}). Because SKJS emits no code, checked programs run unchanged on a JavaScript engine, and the corresponding registry claims must hold in ECMAScript itself. We therefore define a realization map from the model's values to ECMAScript language values and show that the model's predicates agree with `IsStrictlyEqual`, `ToBoolean`, `typeof`, `IsCallable`/`IsConstructor`, and `IsArray`. This establishes **L-TAG**, **L-ISARR**, and **L-INST** subject to their named environment assumptions. It also relates the checker's four-point `Callability` lattice to ECMAScript call and construct behavior, including constructor values (**A-CTOR-CALL**). The remaining `typeof "function"` problem is not a mismatch with ECMAScript: subtyping can admit a callable into a type marked non-callable, as recorded by **F-CABSENT**.

Contents

1 Purpose and Relation to the Main Paper **2**

2 Minimal Recap of the Modeled Runtime **2**

3 The Realization Map: Tags vs. ECMAScript Language Types **3**
3.1 The `typeof` table . . . 4
3.2 Where the `void` tag fits . . . 5

4 Strict Equality: $\approx$ is the Pullback of `IsStrictlyEqual` **5**

5 Truthiness: the Falsy Sets are `ToBoolean` **6**

6 Callable and Constructible **6**
6.1 The `Callability` lattice and **A-CTOR-CALL** . . . 7

7 `instanceof` and L-INST **8**
7.1 The spec algorithms . . . 8
7.2 When the checker narrows and which type it constructs . . . 8
7.3 Discharge . . . 9

8 `Array.isArray` and L-ISARR **10**
8.1 The spec side 10
8.2 The model side: how `ReadonlyArray` acquires the brand 10
8.3 Discharge 11

9 Status and Ledger Edits **12**

A Implementation and Specification Anchors **14**

1 Purpose and Relation to the Main Paper

This document is deep dive A of the series supporting *Refinement and Inference Growth in SKJS: a Soundness Model of the Type Checker* (the *main paper*), pinned like it to revision `bafdcf7c`. Its job is to discharge, sharpen, or refute the rows of the main paper's assumption ledger (its Section "Assumptions and Proof Obligations") that concern the *runtime* rather than the checker:

- **L-TAG** — every value carries exactly one tag, consistently with its static tag. *SKJS emits no code*: a checked program is executed, byte for byte, by an ordinary JavaScript engine, so the runtime counterpart of the model's Lemmas A0/A1 is a statement about ECMAScript's language types, provable from the specification (Section 3) — which is why the ledger anchors this row to ECMAScript rather than to any SKJS component.
- **L-INST** — $P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$ for the instance type built at an `instanceof` narrowing site (Section 7).
- **L-ISARR** — $\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$ (Section 8).

In addition it grounds two definitions the main paper introduces without external justification and then uses everywhere: the strict-equality relation $\approx$ (its Definition "Strict Equality"; used by R-Eq, R-EqField, and constant removal) and the per-tag truthiness table (its Definition "Truthiness"; used by R-Truthy/R-TruthyField and the residue lemma). Both are shown to be *exactly* the ECMAScript algorithms, transported along the realization map (Sections 4 and 5).

Where a discharge is conditional, the condition is a named registry row — either an environment assumption this paper states (**A-DDA**, **A-PROTO**, **A-PROXY**, **A-HASINST**) or an imported row of another paper cited by its canonical statement (**F-CABSENT**). All spec citations are to ECMA-262 as of the 2025 edition; clause *numbers* drift between editions, so each citation also names the clause.

Notation. We reuse the main paper's notation unchanged: Val, Tags, rtag, payloads $(\mathsf{Payload}_k)_k$, ObjVal, the object interface (props, get, has, callable, constructible, isArrayV), $\approx$, truthy, and $[\![\cdot]\!]$. Implementation anchors (impl: `...`) point into revision `bafdcf7c`.

2 Minimal Recap of the Modeled Runtime

From the main paper's Section 2 (Runtime Model): Tags is the nine-element set

$$\mathsf{Tags} = \{\mathsf{undef}, \mathsf{void}, \mathsf{null}, \mathsf{bool}, \mathsf{num}, \mathsf{str}, \mathsf{bigint}, \mathsf{sym}, \mathsf{obj}\},$$

and $\mathsf{Val} = \{(k, p) \mid k \in \mathsf{Tags}, p \in \mathsf{Payload}_k\}$ with $\mathsf{Payload}_{\mathsf{num}} = \mathbb{F}$ (IEEE-754 doubles including -0 and NaN), $\mathsf{Payload}_{\mathsf{bigint}} = \mathbb{Z}$, and $\mathsf{Payload}_{\mathsf{undef}} = \mathsf{Payload}_{\mathsf{void}} = \{\star\}$. The void tag is declared there to be "a checker fiction for the runtime value `undefined`"; this paper makes that statement precise. The relation $\approx$ is a partial equivalence (NaN irreflexive, $-0 \approx 0$, the $\mathsf{undef}/\mathsf{void}$ pair identified); truthy is given by per-tag falsy sets; objects come with the abstract interface above, read as a snapshot (**L-PURE**).

The three refinement rules whose soundness proofs consume the assumed rows are R-Typeof (via the $P_{\mathsf{typeof}=k}$ sets and the $\mathsf{constructible} \implies \mathsf{callable}$ interface fact), R-Inst (via **L-INST**), and R-IsArray (via **L-ISARR**).

3 The Realization Map: Tags vs. ECMAScript Language Types

ECMAScript defines exactly eight *language types*: Undefined, Null, Boolean, String, Symbol, Number, BigInt, and Object (ECMA-262 §6.1, "ECMAScript Language Types"). Every language value belongs to exactly one of them — the types are introduced as a partition of the value space, not as overlapping classifications. Write ESVal for the set of language values and $\mathsf{estag}(v)$ for the language type of $v \in \mathsf{ESVal}$.

Definition 1 (Realization Map)**.** $\gamma : \mathsf{Val} \to \mathsf{ESVal}$ *sends a model value to the language value it stands for:*

- $\gamma(\mathsf{undef}, \star) = \gamma(\mathsf{void}, \star) =$ ***`undefined`*** *and* $\gamma(\mathsf{null}, \star) =$ ***`null`****;*
- *on* bool*,* num*,* str*,* bigint*: the evident payload realization — the model chose its payload sets to* be *the spec's value sets (*$\mathbb{F}$ *is exactly the Number type, "double-precision 64-bit binary format IEEE 754-2019" including* -0 *and a single* NaN*, ECMA-262 §6.1.6.1;* $\mathbb{Z}$ *is exactly the BigInt type, ECMA-262 §6.1.6.2;* String *realizes as UTF-16 code unit sequences, ECMA-262 §6.1.4);*
- *on* sym *and* obj*: a fixed bijection between the abstract payload sets* SymVal*,* $\mathsf{Payload}'_{\mathsf{obj}}$ *and the runtime symbol/object identities of the executing program.*

Theorem 2 (Grounding of Tags; discharges **L-TAG**)**.** γ *is surjective onto the language values reachable by the program, and:*

(G1) $\mathsf{estag}(\gamma(v))$ *is determined by* $\mathsf{rtag}(v)$*, via the evident bijection* $\mathsf{Tags} \setminus \{\mathsf{void}\} \leftrightarrow \{$*Undefined, Null, Boolean, Number, String, BigInt, Symbol, Object*$\}$*, extended by* $\mathsf{void} \mapsto$ *Undefined.*

(G2) γ *is injective except on the single fiber* $\{(\mathsf{undef}, \star), (\mathsf{void}, \star)\}$*, whose two elements realize to the same value* ***`undefined`****.*

(G3) Distinct tags other than the pair $\{\mathsf{undef}, \mathsf{void}\}$ *realize to disjoint sets of language values.*

Consequently the runtime counterparts of the main paper's Lemmas A0 (tag consistency) and A1 (tag disjointness) hold: a language value inhabits denotations of at most one tag, except that ***`undefined`*** *inhabits both* undef*-* *and* void*-tagged denotations.*

Proof. (G1) and (G2) are immediate from Definition 1. (G3) follows from (G1)+(G2) because the eight language types partition ESVal (ECMA-262 §6.1). Surjectivity: every language type is covered, and the payload realizations are onto their value sets (bijections or identity). □

The residual obligation created by (G2) is that no rule of the checker may depend on *runtime-distinguishing* the tags undef and void — they are indistinguishable after realization. The main paper already maintains this discipline explicitly: R-EqUndef and the `typeof "undefined"` case of R-Typeof treat the two tags jointly; the truthiness residue keeps void on both branches; the one corner where a single tag of the pair could be consumed alone is closed there by the checker's rejection of `void`-typed strict-equality operands. Nothing further is needed here.

Anchoring. There is no code generator anywhere in the pinned revision; the runtime is whatever engine executes the unmodified source, so the ledger's L-TAG row is anchored to this theorem (equivalently, to ECMA-262 §6.1), not to any SKJS component. Note the row's phrase "consistent with its static tag" asks, at full strength, that checked expressions inhabit their checked types; that is **L-TC**, a different row, and this paper deliberately does not fold it back in.

3.1 The `typeof` table

The `typeof` operator (ECMA-262 §13.5.3, "The `typeof` Operator", Table "typeof Operator Results") maps every language value to a string as follows, where *callable* means "implements the [[*Call*]] internal method":

Language type of v	`typeof` v	model tag(s)
Undefined	`"undefined"`	undef, void
Null	`"object"`	null
Boolean	`"boolean"`	bool
Number	`"number"`	num
String	`"string"`	str
BigInt	`"bigint"`	bigint
Symbol	`"symbol"`	sym
Object, not callable	`"object"`	obj
Object, callable	`"function"`	obj

Two quirks are necessary for the checker and both are already encoded in the main paper's $P_{\mathsf{typeof}=k}$ sets (its Definition "Evaluation of Condition Forms"): `typeof null === "object"` (the `null` row above; the model includes $[\![\texttt{null}]\!]$ in $P_{\mathsf{typeof}=\texttt{"object"}}$), and the callable/non-callable split of the Object type (the model's $P_{\mathsf{typeof}=\texttt{"function"}} = \{o \in \mathsf{ObjVal} \mid \mathsf{callable}(o)\}$). Granting Definition 6 below (callable = implements [[*Call*]]), the pullback along γ of "`typeof` $x = k$" is *exactly* $P_{\mathsf{typeof}=k}$ for all eight operands. So the evaluation semantics of R-Typeof is grounded; the soundness of its narrowing given that semantics is the main paper's Lemma "R-Typeof is Branch-Sound" and is not re-proved here. (How the checker's *types* populate the callable split is Section 6.1.)

Named environment caveat (A-DDA). Annex B of the spec (ECMA-262 §B.3.6, "The [[*IsHTMLDDA*]] Internal Slot") sanctions host objects — in practice the single browser object `document.all` — for which `typeof` returns `"undefined"` and `ToBoolean` returns false despite being objects. Such a value would break rows one and eight of the table and the truthiness grounding below. We record **A-DDA**: *no* [[*IsHTMLDDA*]] *object flows into checked code.* This holds in any non-browser embedding (Node has none), and the pinned import stubs describe a Node-style environment (`process`, `fetch`/undici — no DOM).

impl: `import-stubs/@types/skjs/index.d.ts`

3.2 Where the `void` tag fits

The checker distinguishes `void` from `undefined` to restrict *programs* (a `void`-typed result should not be consumed), not to describe a runtime distinction; Theorem 2 (G2) is the formal statement that no runtime distinction exists. The grounded reading of every void-related clause in the main paper is obtained by applying γ: e.g. the cross-tag arm of $\approx$ (next section) is not a modeling convenience but a *theorem forced by* $\gamma(\mathsf{undef}, \star) = \gamma(\mathsf{void}, \star)$.

4 Strict Equality: $\approx$ is the Pullback of `IsStrictlyEqual`

The spec's strict-equality algorithm (ECMA-262 §7.2.16, $\texttt{IsStrictlyEqual}(x, y)$; the semantics of `===` via ECMA-262 §13.11) is: (1) if $\mathsf{estag}(x) \neq \mathsf{estag}(y)$, return false; (2) if x is a Number, return $\texttt{Number::equal}(x, y)$ — which returns false if either argument is NaN, true if both are $+0/-0$ in any combination, and value equality otherwise (ECMA-262 §6.1.6.1.13); (3) otherwise return $\texttt{SameValueNonNumber}(x, y)$ — payload equality per type: trivially true for Undefined and Null, mathematical-value equality for BigInt, code-unit equality for String, identity for Boolean/Symbol/Object (ECMA-262 §7.2.12).

Theorem 3 (Grounding of $\approx$)**.** *For all $v, w \in \mathsf{Val}$: $v \approx w$ iff $\texttt{IsStrictlyEqual}(\gamma(v), \gamma(w)) = \mathsf{true}$.*

Proof. Case on the tag pair. If $\mathsf{rtag}(v), \mathsf{rtag}(w) \in \{\mathsf{undef}, \mathsf{void}\}$: both sides realize to `undefined`; step (1) passes, step (3) returns true; and $v \approx w$ by the first arm of the definition — both sides hold. If the tags are equal and not in the pair: realization preserves the language type, step (1) passes, and steps (2)/(3) compute exactly the payload condition of $\approx$'s second and third arms (NaN-irreflexivity and $-0 = 0$ from `Number::equal`; payload/identity equality from `SameValueNonNumber` — for BigInt this is mathematical equality and $\mathsf{Payload}_{\mathsf{bigint}} = \mathbb{Z}$ has no signed zero, so no analogue of the -0 clause arises). If the tags differ and are not the $\{\mathsf{undef}, \mathsf{void}\}$ pair: by Theorem 2 (G3) the realizations have different language types, step (1) returns false, and $\approx$ does not relate v, w — both sides fail. □

Every property of $\approx$ that the main paper proves or uses — the partial-equivalence lemma, $\mathsf{NaN} \not\approx \mathsf{NaN}$, the identification of the undef/void pair — is therefore inherited from the spec rather than stipulated.

Contrast: `SameValue` and `Object.is`. The spec has two finer equalities: `SameValue` (ECMA-262 §7.2.10; surfaced as `Object.is`) makes NaN reflexive and *separates* $+0$ from -0; `SameValueZero` (ECMA-262 §7.2.11; used by `Array.prototype.includes`, `Map`/`Set` keying) makes NaN reflexive and identifies the zeros. Neither is $\approx$. This matters in two directions: (i) the checker narrows only on `===`/`!==`, so only `IsStrictlyEqual` needs grounding — `Object.is` is not even in the pinned stubs, and `Map`/`Set`/`includes` are opaque signatures (their behavior is governed by subtyping); (ii) inside `OrdinaryHasInstance` the spec compares prototype-chain entries with `SameValue`, but on Objects `SameValue` and `IsStrictlyEqual` coincide (both are identity), so no discrepancy reaches Section 7.

Corollary 4 (The literal-type closure choice is forced)**.** *The main paper defines literal-type denotations as same-tag $\approx$-closures, so the numeric literal type `0` denotes $\{+0, -0\}$. Under Theorem 3 this is the unique sound choice for* R-Eq*'s false branch: `x !== 0` does not exclude $x = -0$ at runtime (-0 `===` `0` is true), so a denotation of `0` omitting -0 would let constant removal subtract a value still possible. It also matches `tsc`, which has no -0 literal type: `const x: 0 = -0` is accepted there, i.e. `tsc`'s literal type `0` likewise contains both zeros.*

5 Truthiness: the Falsy Sets are `ToBoolean`

`ToBoolean` (ECMA-262 §7.1.2) is defined by a per-type table: `undefined` and `null` ↦ false; Boolean ↦ itself; Number ↦ false iff the value is $+0$, -0, or NaN; String ↦ false iff empty; BigInt ↦ false iff 0; Symbol ↦ true; Object ↦ true (with the Annex-B $[[IsHTMLDDA]]$ exception covered by **A-DDA**).

Theorem 5 (Grounding of Truthiness)**.** *Under* ***A-DDA****, for all* $v \in \mathsf{Val}$*:*

$$\mathsf{truthy}(v) = \mathit{ToBoolean}(\gamma(v)).$$

Proof. Row-by-row comparison of the main paper's Definition "Truthiness" with the table above, through the tag correspondence of Theorem 2: the undef, void rows both realize to `undefined` (falsy — the model marks both tags falsy, consistently); the numeric falsy set $\{0, -0, \mathsf{NaN}\}$, the empty string, the zero bigint, and the always-truthy symbol/object rows are verbatim. The comparison is exact and complete: neither table has a row the other lacks. □

This grounds the predicate. Note where the burden then sits: the residue lemma of the main paper (its Lemma "Residues are Sound") additionally needs the *implementation's lexeme-level* falsiness test to agree with truthy on literal spellings — a checker-internal (not runtime) lexeme-level matter, tracked in the registry under literal canonicalization and out of scope here. The runtime side contributes no further caveat.

impl: `TType.sk / TPrimValueKind::isFalsy, SType::falsiness`

6 Callable and Constructible

Definition 6 (Grounded predicates)**.** *For* $o \in \mathsf{ObjVal}$*:* $\mathsf{callable}(o)$ *iff* $\gamma(o)$ *implements the* $[[Call]]$ *internal method (the spec's* `IsCallable`*, ECMA-262 §7.2.3);* $\mathsf{constructible}(o)$ *iff* $\gamma(o)$ *implements* $[[Construct]]$ *(*`IsConstructor`*, ECMA-262 §7.2.4).*

"Callable" therefore means *has the internal method*, not "can be invoked without throwing". This distinction is exactly what the `typeof` table keys on, and it is where class constructors live: a class constructor is an ECMAScript function object whose $[[Call]]$ *exists but throws* — step "If $F.[[IsClassConstructor]]$ is true, throw a `TypeError`" in the $[[Call]]$ of ECMAScript function objects (ECMA-262 §10.2.1). Hence `typeof (class {}) = "function"`.

Theorem 7 (Interface fact: constructible $\implies$ callable)**.** *For every* $o \in \mathsf{ObjVal}$*,* $\mathsf{constructible}(o)$ *implies* $\mathsf{callable}(o)$*.*

Proof. The spec states it directly in the definition of the Object type (ECMA-262 §6.1.7.2, "Object Internal Methods and Internal Slots"): "every object that supports $[[Construct]]$ must support $[[Call]]$; that is, every constructor must be a function object" (reiterated among the invariants of the essential internal methods, ECMA-262 §6.1.7.3). It is also easy to audit exhaustively across the function kinds the spec defines: *ordinary functions* — $[[Construct]]$ is installed only on function objects (arrow functions and methods get $[[Call]]$ only); *class constructors* — both present, $[[Call]]$ throwing (ECMA-262 §10.2.1); *bound functions* (ECMA-262 §10.4.1) — always have $[[Call]]$, and $[[Construct]]$ only if the bound target has it; *Proxy exotic objects* (ECMA-262 §10.5.12–§10.5.13) — a proxy has $[[Call]]$ iff its target does and $[[Construct]]$ iff its target does, so the implication lifts through proxies by induction on the target chain. □

This discharges the "JS fact" quoted in the main paper's Object Interface (its Definition "Object Interface") and used in the `"object"` case of its Lemma "R-TYPEOF is Branch-Sound": status PROVED.

6.1 The `Callability` lattice and A-CTOR-CALL

The checker records, per object branch, a callability mode and a constructibility mode drawn from the four-point lattice `Callability` (`TType.sk:2189-2237`), ordered $\mathsf{CPresent}(\sigma) \leq \mathsf{CFunction} \leq \mathsf{CUnknown}$ and $\mathsf{CAbsent} \leq \mathsf{CUnknown}$. Its grounded readings, for a branch B and every $o \in [\![B]\!]$:

- $\mathsf{CPresent}(\sigma)$: $\mathsf{callable}(o)$ (resp. $\mathsf{constructible}(o)$) with invocation signature σ;
- $\mathsf{CFunction}$: $\mathsf{callable}(o)$ (resp. $\mathsf{constructible}(o)$), no signature known — invocation is still rejected (`maybeGetCall` requires $\mathsf{CPresent}$), so the mode is a pure *predicate* claim;
- $\mathsf{CAbsent}$: $\neg\mathsf{callable}(o)$ (resp. $\neg\mathsf{constructible}(o)$);
- $\mathsf{CUnknown}$: no claim.

Two constructions keep the modes consistent with the runtime facts of this section. First, the constructible-implies-$[[Call]]$ invariant: whenever a branch acquires $\mathsf{new} \in \{\mathsf{CPresent}, \mathsf{CFunction}\}$ with $\mathsf{call} = \mathsf{CUnknown}$, construction forces $\mathsf{call} = \mathsf{CFunction}$ (`TType.sk:2273-2281`) — the static image of Theorem 7. Second, the constructor-value typing:

Theorem 8 (Constructor-value modes; discharges **A-CTOR-CALL**)**.** *The static-side object type of a class C (the type of the constructor* value*) carries* $\mathsf{new} = \mathsf{CPresent}(\sigma_C)$ *for the declared (or synthesized empty) construct signature and* $\mathsf{call} = \mathsf{CFunction}$ *(`TEnv.sk:1042-1057`). Both claims hold of the runtime value: $\gamma(C)$ implements $[[Construct]]$ with the declared signature (it is the class's constructor function object) and implements $[[Call]]$ (ECMA-262 §10.2.1 — the $[[Call]]$ exists and throws), so `typeof C === "function"` and the checker's modes assert exactly the true predicates. That no call* signature *is offered is forced: a class constructor's $[[Call]]$ unconditionally throws, so* $\mathsf{CFunction}$ *("callable, not invocable") is the strongest sound mode.*

The `typeof-"function"` narrowing reads the call mode alone — sound by the invariant above — via `narrowByTypeofFunction` (`TType.sk:2438-2460`): the true branch keeps $\mathsf{CPresent}/\mathsf{CFunction}/\mathsf{CUnknown}$ branches (strengthening a surviving $\mathsf{CUnknown}$ to $\mathsf{CFunction}$, justified by the runtime proof of $[[Call]]$) and drops $\mathsf{CAbsent}$ branches; the false branch drops the provable function branches and stamps survivors $\mathsf{CAbsent}$. By Theorem 8 a constructor value survives the true branch with its construct signature intact — both runtime facts are represented, so neither branch excludes a value it can hold.

The one caveat is the $\mathsf{CAbsent}$ drop itself: it is sound exactly when the mode's grounded reading ($\neg\mathsf{callable}$ of every member) is trustworthy. $\mathsf{CAbsent}$ is assigned both where it is derivable (object literals, enum objects, the array-brand lowering of `TType.sk:2266-2272`) and as a *brand* on nominal class-instance types (`TEnv.sk:1000-1008`), which structural subtyping does not validate: the registry row **F-CABSENT** (imported here; owner deep dive D) records that the lax $(\mathsf{CUnknown}, \mathsf{CAbsent})$ subtyping arm admits callables into branded types, and is VIOLATED — witnessed end-to-end (issue report `issues/cabsent-brand-laundering.md`). Within this paper's scope the import is consumed only here: the `"function"`-true drop and the `"object"`-true keep of a $\mathsf{CAbsent}$ branch are sound *given* F-CABSENT's statement, and unsound where its violation is exercised.

impl: `TType.sk / Callability, narrowByTypeofFunction; TEnv.sk / newStaticObject; TypeCheckRefine.sk / evalTypeofEquality`

7 `instanceof` and L-INST

7.1 The spec algorithms

Evaluation of `V instanceof T` calls `InstanceofOperator`(V, T) (ECMA-262 §13.10.2):

1. if T is not an Object, throw;
2. let h = `GetMethod`$(T,$ `@@hasInstance`$)$; if h is not `undefined`, call h with receiver T and argument V, and return the `ToBoolean` of the result;
3. otherwise, if T is not callable, throw; return `OrdinaryHasInstance`(T, V).

The default `@@hasInstance` method, on `Function.prototype` (ECMA-262 §20.2.3.6), exists on every function and simply runs `OrdinaryHasInstance`; it is non-writable and non-configurable, but a class can *shadow* it with a static method, and any object can carry an own `@@hasInstance`. `OrdinaryHasInstance`(C, O) (ECMA-262 §7.3.22): if C is a bound function, recurse on its target; if O is not an Object, return false; let P = `Get`$(C,$ `"prototype"`$)$ (throw if not an Object); walk O's $[[GetPrototypeOf]]$ chain, returning true iff some entry is `SameValue`-identical to P.

Ground $P_{\mathsf{inst}}(C) \triangleq \{v \in \mathsf{Val} \mid$ `InstanceofOperator`$(\gamma(v), \gamma(C)) =$ true$\}$; the throwing cases satisfy neither branch relation (main paper, Definition "Evaluation of Condition Forms"), so they are vacuous for narrowing. That $P_{\mathsf{inst}}(C) \subseteq \mathsf{ObjVal}$ is immediate from step 3 of `OrdinaryHasInstance` — *provided* evaluation reaches it, which step 2 does not guarantee: the analysis below therefore splits on whether a non-default `@@hasInstance` can govern the test (the A-HASINST question).

7.2 When the checker narrows and which type it constructs

R-Inst fires only on the true branch, and only when the RHS is a bare name that resolves to a class or interface *definition* (`hasClassDef`) *and* whose static type declares no custom `[Symbol.hasInstance]` member, directly or inherited (`rhsDeclaresHasInstance`, `src/TypeCheckRefine.sk:448-470`; the conjunction at `src/TypeCheckRefine.sk:506-511`). A merely Function-typed RHS narrows nothing, and a class that overrides the runtime membership test narrows nothing either — the override decides membership and may accept non-instances, so suppression is the sound response (`tsc` narrows there anyway, unsoundly; TS parity is deliberately given up).

The instance view is built with *skolem* type arguments (`makeSkolemTArgsOf`, `src/TEnv.sk:780-808`): one synthetic rigid type parameter per class parameter — printed `Box#T`, a spelling no source type reference can collide with — carrying exactly the parameter's normalized declared constraint. Each represents a type argument *fixed by the runtime object but hidden from the program*: it is never an inference candidate, so no later use can bind it (contrast `makeTArgsOf`, `src/TEnv.sk:822-840`, the inference-side constructor, used here only inside a throwaway probe clone). Narrowing then proceeds per branch of the scrutinee (`evalInstanceof`, `src/TypeCheckRefine.sk:362-440`):

- a branch that already satisfies the instance shape — decided by an `isSub` probe against a fresh-candidate instantiation in a cloned substitution, so the real state is untouched — is kept *unchanged*, preserving the type arguments the scrutinee already carries;
- any other branch is intersected with the skolem view, and the scrutinee's readonly fields are restored afterwards (`keepReadonlyFrom`, `src/TypeCheckRefine.sk:407-416`) — the class view's fields are mutable and intersection unions set-types, so without the restoration a narrowing would grant writes the value's type withheld;

- a scrutinee with no object branch falls back to the skolem view itself.

impl: `TypeCheckRefine.sk / evalCond (InstanceOf arm), evalInstanceof, rhsDeclaresHasInstance; TEnv.sk / hasClassDef, makeSkolemTArgsOf`

7.3 Discharge

The remaining A-HASINST case. The check rules out the declared case: a custom `[Symbol.hasInstance]` in a class body resolves into the static side's symbol fields (the `FNComputed` arm of `addFields`; `Symbol.hasInstance` is a stubbed `unique symbol` found via `getWellKnownSymbolUniqueId`), and `rhsDeclaresHasInstance` searches every branch of the RHS's static type for that symbol key, inherited members included, before narrowing is allowed. What no static check can see is a hook installed *dynamically* on an RHS whose declaration omits it — through the unchecked boundary (ambient `declare`d functions), since the checked language offers no route: the stubs declare no `Object.defineProperty` and no `Proxy`/`Reflect`. The registry row **A-HASINST** records exactly this remaining case, status ASSUMED, alongside **A-PROTO**'s stub discipline.

impl: `TypeCheckRefine.sk / rhsDeclaresHasInstance; TEnv.sk / addFields (FNComputed), getWellKnownSymbolUniqueId`

Theorem 9 (Discharge of **L-INST**). *Assume, for the class C at a narrowing site:*

- ***A-HASINST**: no `@@hasInstance` method other than the `Function.prototype` default is found on $\gamma(C)$ (step 2 resolves to the default; the declared case is excluded by the check, the dynamic case by the assumption);*
- ***A-PROTO**: no prototype surgery — objects with C.`prototype` in their chain were produced by `new` on C or a subclass (no `Object.create`, `Object.setPrototypeOf`, `Reflect`, or Proxy $[[GetPrototypeOf]]$ traps reachable), and C.`prototype` itself is unmodified;*
- ***L-TC** at construction: every `new`-site establishes the instance object type of the constructed class (fields present and typed, per the class-checking discipline).*

*Let T_C be the installed type at the narrowing site — per branch, either the unchanged scrutinee branch, or its intersection with the skolem view $C\langle\vec{s}\rangle$ (readonly restored), or $C\langle\vec{s}\rangle$ itself — and let ρ' be the post-narrowing state. Then $P_{\mathsf{inst}}(C) \cap [\![B]\!]_\rho \subseteq [\![B']\!]_{\rho'}$ for every scrutinee branch B with narrowed form B', i.e. **L-INST** holds for the implementation's installed type.*

Proof sketch. Let $v \in P_{\mathsf{inst}}(C)$. By **A-HASINST**, membership was decided by `OrdinaryHasInstance`, so v is an object with C.`prototype` in its chain. By **A-PROTO**, v was constructed by C or a subclass, as some instantiation $C\langle\vec{U}\rangle$ with $\vec{U}$ within the declared bounds. By **L-TC** at the construction site, v satisfies the instance object type of $C\langle\vec{U}\rangle$: every declared field present with a $\vec{U}$-instantiated get-type, inherited fields included (subclass constructors must complete `super()`).

Per branch, at the post-narrowing state ρ': a *kept* branch is unchanged — the narrowing installs $B' = B$, and no binding of the real substitution is touched in deciding to keep it (the probe runs in a discarded clone), so $[\![B]\!]_{\rho'} = [\![B]\!]_\rho \ni v$. An *intersected* branch has $B' = (B \sqcap C\langle\vec{s}\rangle)$ with readonly restored: the skolems $\vec{s}$ are read under the per-view witness assignment of imported row **SKOLEM-SEM** — each denotes the runtime object's actual hidden argument U_i — so $v \in [\![C\langle\vec{s}\rangle]\!]$ by the previous paragraph, and $v \in [\![B]\!]_\rho$ by hypothesis, so v is in the semantic intersection. Two cases on the implemented result. If $\texttt{inter}(B, C\langle\vec{s}\rangle) \neq \bot$, the imported **L-INTER** non-bottom containment puts v in the installed branch. If the result *is* $\bot$, the branch is dropped — sound only if the operands are semantically disjoint, which contradicts v inhabiting both; that is precisely the

imported disjointness clause **L-INTER-BOT**, VIOLATED, so this case is a named condition of the discharge, live exactly where the operand types can reach the violated strata (function-signature and beyond-precision numeric content nested in the branch). Restoring the scrutinee's readonly fields only lowers set-types, which get-only membership does not consult. The fallback branch is the middle case with $B = \top_{\mathsf{obj}}$. No inference candidate is created in the installed type — the probe's candidates live in a discarded clone — so the skolems can never be bound by later uses: the interpretation $s_i := U_i$ is a fixed parameter of the installed type's denotation, not open inference state, and survival of the judgement under later growth is the general transport question (row **OBL-MONOTONE**, imported), not an **L-INST**-specific caveat. Mutation after construction is governed by **L-MUT** exactly as for every other membership statement in the main paper; the write permissions of the installed views are analyzed by deep dive H (row **L-MUT-ALIAS**). □

A-PROTO audit. The pinned stubs deliberately omit APIs that modify prototypes: the `Object` statics `create`, `getPrototypeOf`, and `setPrototypeOf` are present only as comments marked "disallowed", and neither `Proxy` nor `Reflect` is declared at all. A checked program therefore cannot express prototype surgery; what remains is the generic unchecked boundary (ambient `declare`d functions, `JSON.parse<T>`) through which any assumption, not specially this one, can be voided — that boundary is priced into **L-TC** already. Status: ASSUMED, adequately enforced for checked code.

impl: `import-stubs/@types/skjs/index.d.ts (Object statics)`

Cross-realm objects. `instanceof` is realm-sensitive: an array (or class instance) from another realm does not have *this* realm's `C.prototype` in its chain, so the operator answers false. For soundness of R-INST this is *benign*: the rule narrows only the true branch, and cross-realm values make the test false, landing in the untouched (identity) false branch. Realms are purely a *completeness* phenomenon for `instanceof` — the classical reason `Array.isArray` exists, as the next section exploits. No realm assumption is needed for **L-INST**; we record this so the ledger does not grow a vacuous row.

8 `Array.isArray` and L-ISARR

8.1 The spec side

`Array.isArray` (ECMA-262 §23.1.2.2) is `IsArray` (ECMA-262 §7.2.2): return true if the argument is an *Array exotic object* (ECMA-262 §10.4.2: the objects with the special `length`-coupled $[[DefineOwnProperty]]$); if it is a Proxy, recurse on the Proxy's target (throwing a `TypeError` if the proxy has been revoked); otherwise false. Three consequences: (i) the test is *cross-realm-safe* — an array from any realm is an Array exotic object, no prototype identity involved — in contrast to `x instanceof Array`; (ii) it is *Proxy-transparent* — a proxy over an array answers true; (iii) on a revoked proxy the *call throws*, so by the main paper's evaluation convention neither branch relation holds and narrowing is vacuously sound for that case. Ground $\mathsf{isArrayV}(o) \triangleq \mathtt{IsArray}(\gamma(o))$ (defined, non-throwing case).

8.2 The model side: how `ReadonlyArray` acquires the brand

L-ISARR claims $\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$. Two things could break this: the stub type could demand a *property* real arrays do not have, or a declared member could fail on real

arrays. At first, the stub appears to require such a runtime field because it spells the brand as an ordinary declaration,

```
declare interface ReadonlyArray<T> extends Iterable<T> {
  readonly __SKJS_isArray: true;
  ...
}
```

and a required field `__SKJS_isArray` would exclude every real runtime array (they carry no such property), making $[\![\mathsf{ReadonlyArray}]\!]$ nearly empty and **L-ISARR** false. The implementation avoids this by construction: `TFields::create` *intercepts* the field named by `kArrayBrandFieldName = "__SKJS_isArray"`, removes it from the field map, and transfers its literal value (`true`/`false`, anything else is a fatal SKJS2051) into the dedicated `TFields.isArray : ?Bool` component — the model's brand isArr, whose membership semantics constrains $\mathsf{isArrayV}$, not props. The brand never exists as an ordinary field of any type built through `TFields::create`, which is the path all class/interface instantiation takes (`TEnv::newObject`). Defaults are set soundly at the same spot: interfaces get $\mathsf{isArr} = \mathsf{None}$ (unknown) and classes get $\mathsf{Some}(\mathsf{false})$ *before* the intercept — so a class inheriting the brand field (e.g. `Array` itself, tuples via `SKJS_Tuple`, or a user `class D extends Array`) still ends at $\mathsf{Some}(\mathsf{true})$, because the inherited brand field is in the collected map and the intercept overrides the default. (Object *literals* take `createFromSorted`, which does not intercept; a user field literally named `__SKJS_isArray` on a literal stays an ordinary — truthfully present — property, which is sound.)

impl: `TType.sk / kArrayBrandFieldName, TFields::create (brand intercept)`

impl: `TEnv.sk / newObject (isArray defaults)`

impl: `import-stubs/@types/skjs/index.d.ts / ReadonlyArray`

8.3 Discharge

Theorem 10 (Discharge of **L-ISARR**)**.** *Assume* ***A-PROXY****: no Proxy exotic object flows into checked code. Then* $\mathsf{isArrayV}(o)$ *implies* $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$.

Proof. Under **A-PROXY**, $\gamma(o)$ is an Array exotic object of some realm. $[\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$ has a single object branch whose constraints we check against the object interface grounded as: $\mathsf{has} =$ `HasProperty` (prototype chain included — this matches both the `in` operator and property reads) and $\mathsf{get} = [[\mathit{Get}]]$, snapshots per **L-PURE**.

- *Brand*: $\mathsf{isArr} = \mathsf{Some}(\mathsf{true})$ (intercepted as above) requires $\mathsf{isArrayV}(o)$ — the hypothesis.
- ***length***: required, get-type `number`. Every Array exotic object owns a `length` whose value is a non-negative Number (ECMA-262 §10.4.2).
- *Numeric index* (`readonly [n: number]: T` at $T = \top$): covers numeric-spelling keys not otherwise named; each such property value lies in $[\![\top]\!] = \mathsf{Val}$. Trivially satisfied (and the $\mathsf{set} = \bot$ readonly encoding is invisible to membership, which is get-only).
- *Methods* (`find`, `every`, `some`, `forEach`, `findIndex`, `join`, `map`, `filter`, `flatMap`, `reduce`, `reduceRight`, `slice` — the full declared list at the pin, several classic members being deliberately commented out for variance reasons): each is a required field whose get-type is a single-branch object with a *definite* call signature (interface methods are built through `TObject::createCall`). Membership requires presence and callability of the property value; the behavioral meaning of the signature itself is opaque and belongs to the main paper's

subtyping claim rather than its object semantics. Every listed name is a function-valued property of `Array.prototype` in every realm (ECMA-262 §23.1.3), reached by `get` through the instance's prototype chain; arrays from other realms find their own realm's identical-by-spec `Array.prototype`. (An own property shadowing a method with a non-callable value would falsify this — that is ordinary property mutation, priced into **L-MUT**/**L-PURE** snapshots like every other field statement, not a new caveat.)

- *`[Symbol.iterator]`* (inherited from the `Iterable<T>` base): same argument; the `@@iterator` method of `Array.prototype` exists in every realm, and well-known symbols are *shared across realms* (ECMA-262 §6.1.5.1), so the checker's unique-symbol key denotes the same runtime symbol everywhere.

- *Extensibility*: all other properties of o are unconstrained by the main paper's extensible object semantics; no string index is declared, so no covering constraint arises.

All constraints hold; o inhabits the branch. □

On **A-PROXY**: a Proxy over an array passes `IsArray` but its [[*Get*]] trap may return arbitrary values for `length` or the methods, falsifying membership; hence the assumption. It is enforced for checked code the same way as **A-PROTO**: `Proxy` is absent from the pinned stubs, so no checked program can construct one, and the unchecked boundary is **L-TC**'s. Note the contrast worth keeping in the ledger note: cross-realm values, the classical failure mode of `instanceof Array`, are handled *correctly* here — no realm assumption is required, thanks to `IsArray`'s realm-independence plus the structural (non-nominal) membership semantics.

The mutability note of the main paper (R-IsArray intersects with *Readonly*Array, never mutable `Array`) needs no runtime grounding: it is a precision choice about which *set-types* to add, and set-types do not participate in membership.

9 Status and Ledger Edits

"Here" means in this deep dive.

Item	**Where**
L-TAG	Theorem 2
≈ grounding	Theorem 3, Corollary 4
Truthiness	Theorem 5
ctor⇒call	Theorem 7
A-DDA	§3.1
A-CTOR-CALL	Theorem 8, §6.1
F-CABSENT (imported)	§6.1
L-INST	Theorem 9
A-HASINST	§7 (the residue paragraph)
A-PROTO	§7
L-ISARR	Theorem 10
A-PROXY	§8

Registry. The authoritative, current statuses of every item above are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-TAG	Every runtime value carries exactly one tag consistent with its static tag.	PROVED
A-CTOR-CALL	Class-constructor values carry CPresent construct and CFunction call modes, consistent with [[Call]]/[[Construct]].	PROVED
L-INST	$P_{\mathsf{inst}}(C) \subseteq [\![T_C]\!]$ for the instance type as instantiated at the narrowing site.	PROVED
L-ISARR	$\mathsf{isArrayV}(o)$ implies $o \in [\![\mathsf{ReadonlyArray}\langle\top\rangle]\!]$.	PROVED
A-PROTO	No prototype surgery / Proxy GetPrototypeOf traps reach checked code (enforced by stub omissions).	ASSUMED
A-DDA	No [[IsHTMLDDA]] object (`document.all`) reaches checked code (vacuous on Node).	ASSUMED
A-PROXY	No Proxy exotic object reaches checked code.	ASSUMED
A-HASINST	No non-default `Symbol.hasInstance` governs a narrowed `instanceof`: narrowing is suppressed when the RHS declares one; dynamically installed hooks are assumed absent.	ASSUMED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:

Row	Canonical statement	Owner	Status
L-INTER	When `inter` returns a non-$\bot$ result, that result contains the semantic intersection of its operands, provided every recursive call that returned $\bot$ had disjoint operands (L-INTER-BOT).	B	OPEN
L-INTER-BOT	A $\bot$ result from `inter` implies semantic disjointness.	B	VIOLATED
L-TC	Checked expressions inhabit their checked types.	main	ASSUMED
L-PURE	Evaluating a condition does not mutate the narrowed locals.	F	VIOLATED
L-MUT	Field writes are governed by set-types; get-only membership is sound for snapshot reads.	main	ASSUMED
L-MUT-ALIAS	A checked field or index write through one mutable view preserves the membership guarantees of every live alias of the same runtime object.	H	VIOLATED
OBL-MONOTONE	Monotone Reads Under Growth: judgements validated under ρ survive later legal transitions.	E	VIOLATED
TVAR-SEM	A type-variable occurrence denotes its substitution envelope intersected with its occurrence constraints; an unbound envelope is empty.	main	PROVED
SKOLEM-SEM	A skolem in an installed `instanceof` view denotes the scrutinee object's actual hidden type argument, fixed at narrowing time and stable thereafter.	main	PROVED
F-CABSENT	The CAbsent brand-like claim accurately excludes callables: refinement drops CAbsent branches under `typeof "function"`, but subtyping's permissive (CUnknown, CAbsent) arm admits callables into branded types.	D	VIOLATED

Method note. One attempted row is worth recording as simply *unnecessary*: a realm assumption for **L-INST** symmetric to the classical `instanceof Array` pitfall does not exist, because true-branch-only narrowing makes realms a pure completeness concern (§7).

A Implementation and Specification Anchors

All implementation references are to revision `bafdcf7c`.

This paper	Anchor
brand name	`TType.sk / kArrayBrandFieldName`
brand intercept	`TType.sk / TFields::create`
brand defaults	`TEnv.sk / newObject`
brand stub spelling	`import-stubs/@types/skjs/index.d.ts`
R-IsArray handler	`TypeCheckRefine.sk / evalIsArray`
array-brand intersection	`TTypeInter.sk / TFields::interIsArray`
R-Inst condition + handler	`TypeCheckRefine.sk / evalCond, evalInstanceof, rhsDeclaresHasInstance`
skolem instance arguments	`TEnv.sk / makeSkolemTArgsOf`; `TTypeSubst.sk / genSkolemTParam`
readonly restoration	`TypeCheckRefine.sk / evalInstanceof (keepReadonlyFrom)`
class-definition check	`TEnv.sk / hasClassDef`
RHS validity (TS2359)	`TypeCheckExpr.sk / isInstanceofRhs`
symbol-keyed class members	`TEnv.sk / addFields (FNComputed)`
static-side class type	`TEnv.sk / newStaticObject`
`Callability` lattice	`TType.sk / Callability, narrowByTypeofFunction`
R-Typeof handler	`TypeCheckRefine.sk / evalTypeofEquality`
truthiness tables	`TType.sk / TPrimValueKind::isFalsy, SType::falsiness`

Specification clauses cited (ECMA-262, 2025-edition numbering; clause names are stable across editions): §6.1 language types; §6.1.5.1 well-known symbols; §6.1.6.1–6.1.6.2 Number/BigInt types (and `Number::equal`, §6.1.6.1.13); §6.1.7.2–6.1.7.3 object internal methods and their invariants; §7.1.2 `ToBoolean`; §7.2.2 `IsArray`; §7.2.3–7.2.4 `IsCallable` and `IsConstructor`; §7.2.10–7.2.12 `SameValue`, `SameValueZero`, and `SameValueNonNumber`; §7.2.16 `IsStrictlyEqual`; §7.3.22 `OrdinaryHasInstance`; §10.2.1 ECMAScript function [[*Call*]] (class-constructor throw); §10.4.1 bound function exotic objects; §10.4.2 Array exotic objects; §10.5.12–10.5.13 Proxy [[*Call*]] and [[*Construct*]]; §13.5.3 `typeof`; §13.10.2 `InstanceofOperator`; §13.11 equality operators; §20.2.3.6 the `@@hasInstance` method of `Function.prototype`; §23.1.2.2 `Array.isArray`; §23.1.3 `Array.prototype` methods; Annex B §B.3.6 [[$IsHTMLDDA$]].